academia global

Universidad Corporativa Coppel | Programas Académicos

Ingeniería en Desarrollo de Software

Actividad | #3 | Programa Banco Mexicano (parte 2) Lenguaje de Programación IV

Ingeniería en Desarrollo de Software

TUTOR: Aarón Iván Salazar Macías.

ALUMNO: Alberth Alexis Rodriguez Astorga

FECHA: 06 de Abril del 2026

Indice

Introducción……...03

Descripción……....03

Justificación……...03

Investigación……..04

Desarrollo……..06

Conclusión……...09

Referencias……...…….09

Introducción

En el desarrollo de sistemas bancarios, es importante que todas las funciones trabajen de manera completa y coordinada, ya que cada operación cumple un papel importante en la experiencia del usuario.

No solo se trata de permitir depósitos y retiros, sino también de ofrecer opciones como consultar el saldo disponible y finalizar la sesión de forma adecuada.

La consulta de saldo es una de las funciones más importantes, ya que permite al usuario conocer en todo momento cuánto dinero tiene disponible, mientras que la opción de salir asegura que el sistema se cierre correctamente, brindando mayor control y seguridad.

Completar estas funciones permite que el programa se acerque más a un sistema real, además de reforzar el uso de la programación orientada a objetos y las estructuras de control, logrando una aplicación más completa, funcional y organizada.

Descripción

La actividad consiste en completar el programa del cajero automático que ya se había comenzado en la etapa anterior.

En este caso, el objetivo es agregar las funciones que faltan para que el sistema esté totalmente funcional, específicamente la opción de consultar el saldo y la opción de salir del programa.

El programa debe mostrar un menú con cuatro opciones: depósito, retiro, saldo y salir.

Cuando el usuario elige la opción de saldo, el sistema debe mostrar en pantalla el monto actual disponible. Por otro lado, si el usuario selecciona salir, el programa debe mostrar un mensaje indicando que la sesión ha sido cerrada.

Además, se debe continuar trabajando bajo el enfoque de la programación orientada a objetos, organizando el código en clases y métodos, y utilizando estructuras como if o switch para manejar las decisiones del usuario.

Justificación

La realización de esta actividad se justifica porque permite completar el desarrollo de un sistema funcional, integrando todas las opciones necesarias para simular el comportamiento básico de un cajero automático.

Al añadir las funciones de consulta de saldo y salida, el programa deja de ser parcial y se convierte en una aplicación más completa y útil.

Además, esta actividad refuerza el uso de la programación orientada a objetos, ya que se requiere organizar correctamente el código en clases y métodos, así como utilizar estructuras de control para manejar las decisiones del usuario.

Esto ayuda a mejorar la lógica de programación y la capacidad de desarrollar soluciones más estructuradas.

Asimismo, el uso de herramientas como Java y el entorno de desarrollo permite trabajar de manera más eficiente, detectar errores y probar el funcionamiento del sistema. En conjunto, la actividad contribuye al desarrollo de habilidades prácticas necesarias para crear programas reales y funcionales.

Investigación

Investiga un ejemplo de cómo declarar una clase abstracta y heredar sus propiedades y métodos a subclases en los siguientes lenguajes de programación:

Una clase abstracta es una clase que no se puede instanciar (no puedes crear objetos directamente de ella) y sirve como una base o "plano" para otras clases. Suele contener al menos un método abstracto, que es un método declarado pero que no tiene implementación, obligando a las subclases a definir su propio comportamiento.

1.Java:

```
abstract class Animal {
   String nombre;

   abstract void hacerSonido();

   void dormir() {
      System.out.println("El animal está durmiendo");
   }
}

class Perro extends Animal {
   void hacerSonido() {
      System.out.println("El perro ladra");
   }
}
```

2.Python

```
from abc import ABC, abstractmethod

class Animal(ABC):
   def __init__(self, nombre):
      self.nombre = nombre

   @abstractmethod
   def hacer_sonido(self):
      pass

class Perro(Animal):
   def hacer_sonido(self):
      print("El perro ladra")
```

3.C#

```
abstract class Animal {
   public string Nombre { get; set; }

   public abstract void HacerSonido();

   public void Dormir() {
      Console.WriteLine("El animal está durmiendo");
   }
}
```

```
class Perro : Animal {
    public override void HacerSonido() {
        Console.WriteLine("El perro ladra");
    }
}
```

4.C++

```
#include <iostream>
using namespace std;

class Animal {
public:
    string nombre;

    virtual void hacerSonido() = 0; // método abstracto

    void dormir() {
        cout << "El animal está durmiendo" << endl;
    }
};

class Perro : public Animal {
public:
    void hacerSonido() {
        cout << "El perro ladra" << endl;
    }
};
```

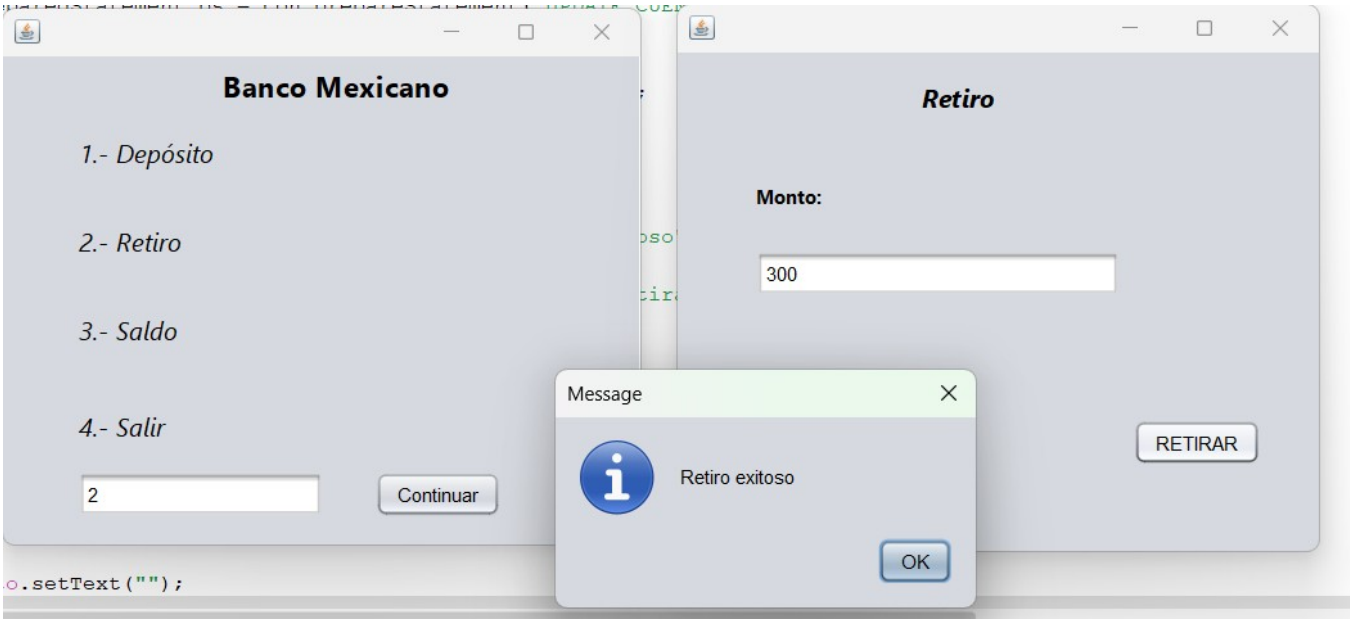
5.PHP

```
<?php
abstract class Animal {
    public $nombre;

    abstract public function hacerSonido();

    public function dormir() {
        echo "El animal está durmiendo";
    }
}

class Perro extends Animal {
    public function hacerSonido() {
        echo "El perro ladra";
    }
}
?>
```

Desarrollo

Continuaremos con la actividad pasada:
Vamos a insertar el código al Retiro.

```java
    private void jbtnRetirarActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        Connection con;
        Conexion conn = new Conexion();

try {
    con = conn.getConnection();
    PreparedStatement ps = con.prepareStatement("UPDATE CUENTA SET saldo = saldo - ? WHERE idcuenta


    ps.setFloat(1, Float.parseFloat(jtfMonto.getText()));

    int res = ps.executeUpdate();

    if (res > 0) {
        JOptionPane.showMessageDialog(null, "Retiro exitoso");
    } else {
        JOptionPane.showMessageDialog(null, "Error al retirar");
    }


    con.close();

} catch (Exception e) {
    System.err.print(e);

}
jtfMonto.setText("");
```



Retiro se hizo de manera exitosa.

SQLQuery1.sql - LA...7RCQL2\alexi (78))*

```sql
select*
from cuenta
```

90 %

Results | Messages

	idcuenta	saldo
1	1	1700

Efectivamente el retiro fue exitoso, de los 2000 que teníamos, nos quedan 1700 por los 300 que retiramos.

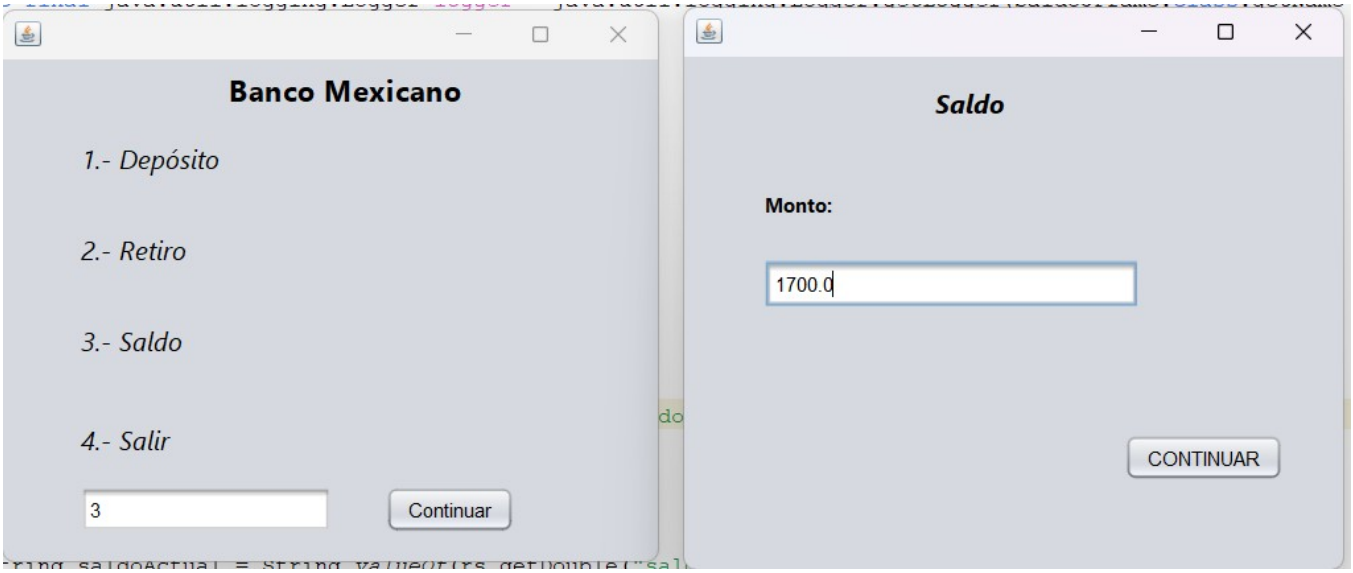
Posteriormente vamos a insertar código en el Saldo:

CajeroJFrame.java × | Main.java × | DepositoJFrame.java × | RetiroJFrame.java × | Conexion.java × | SaldoJFrame.java ×

Source | Design | History

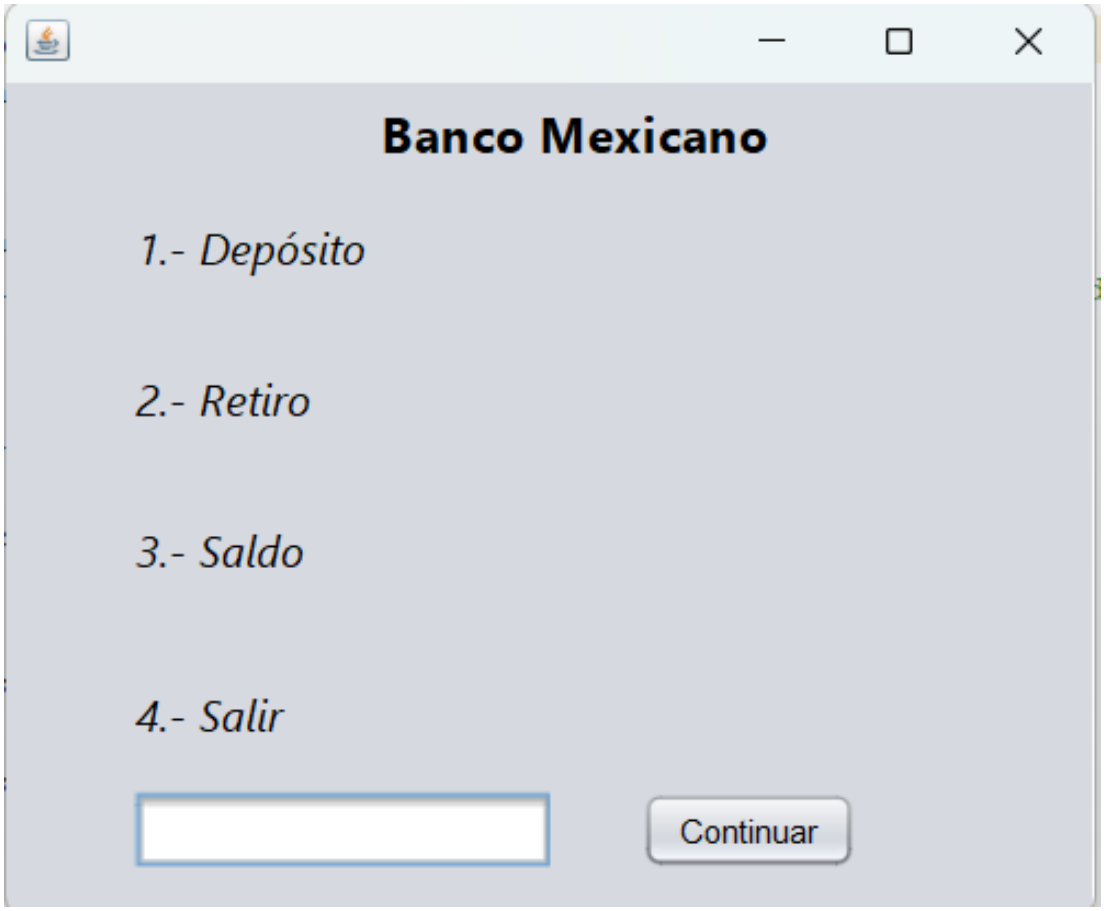
```java
    public SaldoJFrame() {
        initComponents();

        Conexion conn = new Conexion();
        Connection con = null;

        try {
            con = conn.getConnection();

            PreparedStatement ps = con.prepareStatement("SELECT saldo FROM CUENTA WHERE idcuenta = 1");
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {

                String saldoActual = String.valueOf(rs.getDouble("saldo"));
                jtfMonto.setText(saldoActual);
            } else {
                jtfMonto.setText("0.00");
            }

        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Error de SQL: " + e.getMessage());
        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
        } finally {

            try { if (con != null) con.close(); } catch (SQLException ex) {}
        }
```

pkg2026_2_actividad2.SaldoJFrame > SaldoJFrame > try > finally



Nos muestra exitosamente la cantidad que tenemos de saldo en la tabla.

Con esto estaríamos dejando totalmente funcional nuestra aplicación de banco.



Totalmente funcional.

Conclusión

El desarrollo de estas actividades permitió avanzar de manera progresiva en la construcción de un sistema funcional, pasando de conceptos básicos de programación hasta la implementación de un programa más completo que simula el funcionamiento de un cajero automático.

A lo largo del proceso, se integraron elementos como la interfaz, las operaciones de depósito y retiro, la consulta de saldo y el cierre de sesión, logrando una aplicación más cercana a un entorno real.

Asimismo, se reforzaron los fundamentos de la programación orientada a objetos, comprendiendo la importancia de utilizar clases, métodos, herencia y abstracción para organizar el código de manera clara, reutilizable y eficiente.

También se desarrollaron habilidades en el uso de estructuras de control y en la resolución de problemas técnicos, lo cual es esencial en el desarrollo de software.

En conjunto, estas actividades no solo permitieron crear un programa funcional, sino que también sentaron las bases para proyectos más complejos, demostrando cómo la programación puede aplicarse para resolver necesidades reales, como la gestión de operaciones bancarias.

Referencias

Microsoft. (2022). *Microsoft SQL Server 2022* (Versión 16.0) [Software de base de datos]. https://www.microsoft.com/es-mx/sql-server/sql-server-downloads

Apache Software Foundation. (2024). *Apache NetBeans* (Versión 21) [Entorno de desarrollo integrado]. https://netbeans.apache.org/ }

Microsoft. (2024). *Microsoft JDBC Driver for SQL Server* (Versión 12.x) [Controlador de base de datos]. https://learn.microsoft.com/es-es/sql/connect/jdbc/download-microsoft-jdbc-driver-for-sql-server

Código GitHub:
https://github.com/AlexisRodriguez1988/Programaci-n-IV/tree/main